#6 Moreover, this way we can check if reached a minimum point, where gradient is zero and no updated is performed (likely, here we want to pass some tolerance `epsilon`, and stop when the sum of the differences between the old and new positions is smaller than `epsilon`)
#7 At the end of each iteration, update current coordinates.

The code in listing 16.5 is a duplicate of the body of the general-purpose gradient descent method we have previously shown in listing 16.4: while it's still possible to use that method, in this case we have a more specific domain that can allow us some optimization, and overall, I believe, to express more clearly how this algorithm internally works.

For instance, you can see that we never use the cost function, but we only need to be able to compute its partial derivatives (now it should make more sense why, as we had mentioned, we prefer to use the squared distance in the cost function, to avoid square roots).

Speaking of the gradient, the partial derivates can easily be computed using the formula we have provided (or a similar one, if you use a different cost function): it only requires running two for loops over the vertices, so explicit code is not shown here.

It's easy to use this method for a random-restart algorithm: just decide how many attempts you'd like to perform, and run a loop calling method `forceDirectedEmbedding`.

The caveat is that, in this case, we do need an explicit definition of the cost function, because (as shown in listing 16.6) after each call to `forceDirectedEmbedding`, we will have to check the cost of the solution returned, and compare it to the best result so far.

Listing 16.6 Method forceDirectedEmbeddingWithRestart